DA6400 : Reinforcement Learning (Jan-May 2026)

Programming Assignment 3

Deadline: May 8, 2026 11:59 PM

1 Instructions

1. **Start early!**
2. Write complete Python code for this assignment. Include your `requirements.txt` and/or conda yaml file along with a README to reproduce your experiments.
3. Submit a report containing the experimental results and answers to the questions below. The answers must be clear, concise, and to the point. A single line explanation is sufficient if it serves the purpose. Unnecessary verbose can be penalized. The report has a strict limit of 10 pages.
4. The maximum marks without the bonus question is 100. The marks distribution is provided below. 100M $\equiv$ 100 marks.
5. We expect one submission per group of 3 members. This assignment is designed for team effort. Each member must contribute for successful completion of the assignment.
6. Read the full PDF before starting any experiments. Take a note of all the logs to store. Advice: Run for 2 seeds to ensure everything is okay. Then run all.
7. It is advised to use a code editor (e.g., VSCode) to handle the code and experiments.
8. **Please strictly follow the academic code of conduct. Plagiarism will be penalized.**

Note: Unless specified otherwise, run all experiments in this assignment for 15 random seeds/runs. A random seed corresponds to the network initializations and the initial start state of the agent.

Note: Plot average undiscounted return vs. environment timesteps plots for all the experiments (use 20 episode (offline) evaluations every 10K steps). Do **NOT** use episodes on the x-axis (will be penalized). Plot mean performance with confidence intervals. Run the experiments for sufficient duration so as to achieve convergence empirically (could be a near-optimal policy for the relevant cases). However, do not run the experiment for an unnecessarily long period because doing so will make experimentation inefficient. An illustration is provided in Figure 1.

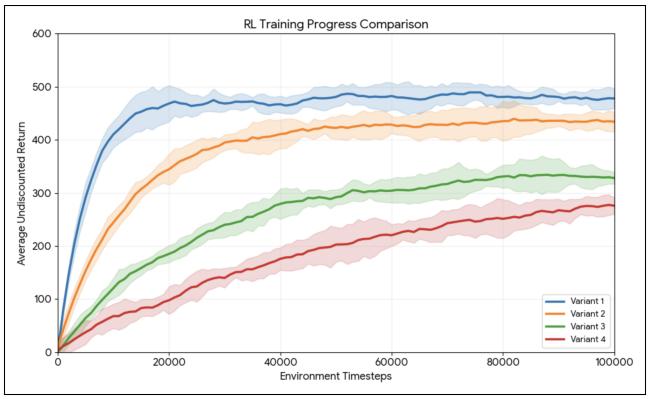
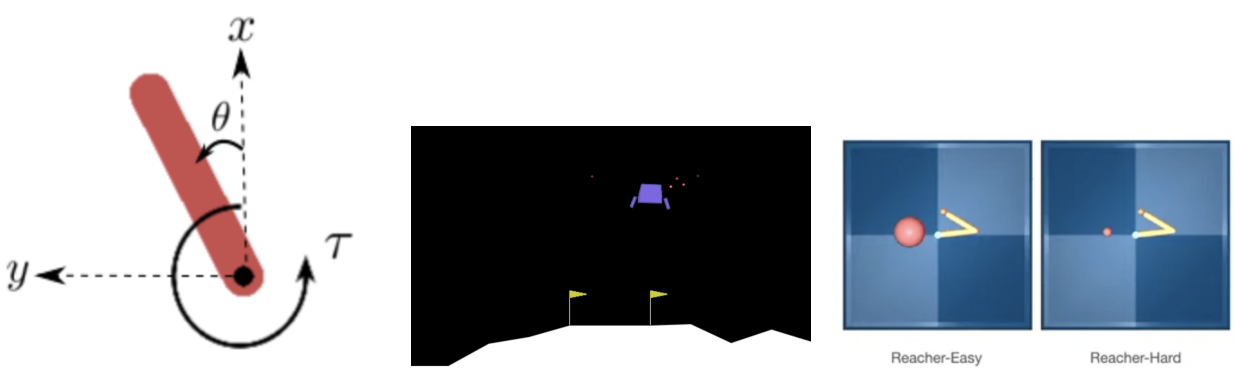


Figure 1: Plot just for illustration (not necessarily representative of the results in this assignment)

2 Soft Actor-Critic

This task is based on implementing and experimenting with the SAC deep RL algorithm. Use the specified environments for training and evaluating your policies. The associated link contains the environment description. Please use the exact version of the environment as specified (Figure 2). Use $\gamma = 0.99$.

Use squashed gaussian policies (tanh). Use a 10K steps random action exploration phase for SAC initially in the training. Use Adam optimizer and clipped double Q-learning for both actor and critic. Use reparameterization trick. Learn only action value function and not the state value function in the SAC implementation.



(a) Pendulum (b) Lunar Lander (c) Reacher

Figure 2: Environments for SAC

2.1 Pendulum

Pendulum-v1 (Figure 2a) is a classic control environment consisting of a pendulum arm attached to a fixed point from one end. The default objective is to swing the pendulum

to an upright position (against gravity) by applying torque on the joint. The pendulum's angle with the vertical is given by $-180 \leq \theta \leq 180$ degrees. The left and the right halves of the 2D space (in which the pendulum swings) correspond to positive and negative angles, respectively, and $\theta = 0$ corresponds to the upright position. **The modified environment objective is to align the pendulum to a given target angle $\theta_{\textbf{target}}$ with the vertical and remaining there.** The environment has no terminal state. The episodes are fixed length and are truncated upon reaching 1000 timesteps.

1. Design and write an appropriate reward function for this environment. **[5M]**

2. Write Python code for SAC and run it on the modified Pendulum-v1. Run for $\theta_{\text{target}} = \{0, -10, 30, -60, 90, -90, 120, -150\}$. Implement the SAC version **discussed in the class** with automated temperature tuning (using target entropy). **[4M]**

3. Compare the curves for different θ_{target}. Compare them in terms of how fast they learn and how well they achieve the desired behavior. **[4M]**

4. What can you say about the optimal behavior? What are the two aspects of it? **[3M]**

5. Implement a SAC version with manual tuning of the temperature parameter α for $\theta_{\text{target}} = \{-60, 90, 120, -150\}$.

   (a) Compare the results for manual tuning vs. automated tuning. How easy/difficult was it to manually tune α? Let us call the found α as α_{mnl}. Note that the optimal α_{mnl} can be different for different target angles. **[5M]**

   (b) Scale the reward by $10\times$ and $0.1\times$. For $\theta_{\text{target}} = 90$, train two SAC versions - (i) SAC with manually tuned α_{mnl} found in part (a) and (ii) SAC with automated tuning. Plot the results. Do (i) and (ii) adapt well to the scaled reward? Why or why not? **[5M]**

2.2 Lunar Lander

LunarLander-v3 (Figure 2b) is a 2D environment focusing on landing a spaceship between two pads. The agent controls a module under simulated gravity and optional wind, attempting a soft landing using main and side engines while balancing fuel consumption. Check the state-action space, reward function, and termination conditions in the documentation. The environment has two versions: discrete and continuous actions.

1. Use the continuous action version of LunarLander. Implement the SAC version **discussed in the class** with automated temperature tuning (using target entropy). **[4M]**

2. Explain the behavior of a SAC agent in the initial, intermediate (can be more than one), and final stages of training. The final stage should correspond to near-optimal policy. Are there local optima in the intermediate stage(s)? Describe them. **[4M]**

3. Consider the following environment variant $\rightarrow$ provide an extra $+200$ reward for hovering, given by a small box in mid-air ($|x| < 0.1$ and $0.4 < |y| < 0.6$). Note that this reward is given just once per episode if the lander comes inside the hover box. All other things in the reward function remain default.

   Take two SAC versions, first with a fixed $\alpha = 0.01$ and second with automated temperature adjustment. Train them on the above environment for a considerable number of timesteps. After training for a while, modify the reward as follows and continue training $\rightarrow$ change the $+200$ reward to -100. Continue training till convergence.

   (a) Compare the quantitative (learning curves) and qualitative behavior of the two SAC versions before and after the reward change ($+200 \rightarrow -100$). Explain if there is any difference between the two versions. **[7M]**

   (b) Keeping this experiment in mind and what was discussed in the class, what can you say about the maximum entropy formulation of SAC? **[7M]**

4. Use the discrete action version of LunarLander.

   (a) Even though SAC is more popularly used for continuous actions, find (or come up with) an appropriate discrete version of SAC. Explain it briefly. **[4M]**

   (b) Implement discrete-SAC in LunarLander and plot the results. **[3M]**

   (c) You had implemented DQN in the previous assignment. Train DQN and compare with discrete-SAC on the discrete-action version of LunarLander. **[3M]**

   (d) In terms of (i) performance and sample efficiency and (ii) ease of implementation, which out of the two algorithms would you prefer for environments with discrete actions? Why? **[4M]**

2.3 Reacher

Consider the reacher environment (Figure 2c) from the deepmind control suite. There are two versions of the environment: easy and hard. You should use the easy version for the experiments in this section. The "reacher" is a robotic arm with two joints. The goal is for the arm to swiftly reach the target sphere and remain there. The target can be any point on the 2D plane.

Figure 3 shows various reward formulations for this environment. The default version uses $\mathcal{R}_b$. However, you need to use all three types of reward formulations for this assignment. You should run the experiments for equal number of timesteps for all three reward formulations. That is, the total number of training timesteps will be determined by the reward formulation that is slowest to converge.

1. Implement the SAC version **discussed in the class** with automated temperature

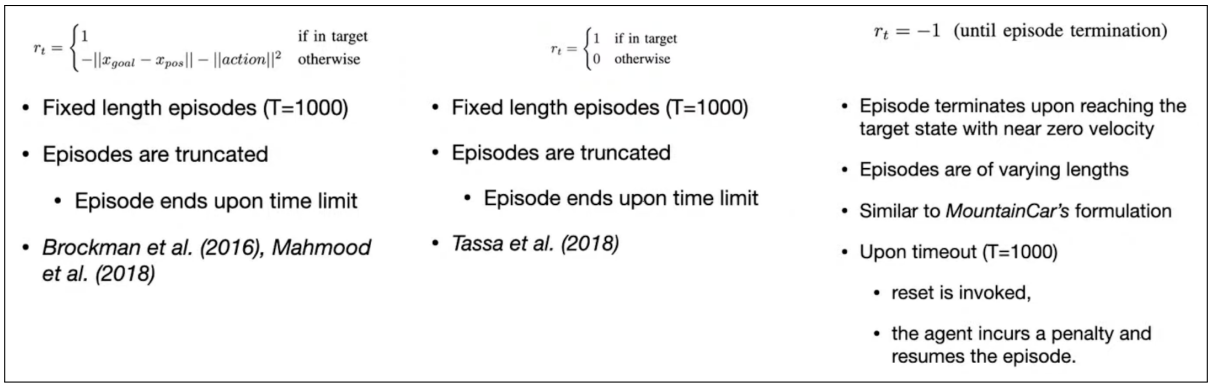


Figure 3: Different reward formulations for reacher. These will be referred to as $\mathcal{R}_a, \mathcal{R}_b, \mathcal{R}_c$ from left to right, respectively. Let us call the SAC agent getting trained using these formulations as SAC-$\mathcal{R}_a$, SAC-$\mathcal{R}_b$, SAC-$\mathcal{R}_c$, respectively.

tuning (using target entropy). Do it on all three reward formulations.

Note: As mentioned previously you need to record the average performance by running 20 (offline) episodes every 10K steps. When performing this evaluation for SAC-$\mathcal{R}_a$, you should evaluate (and log returns for) the policy with respect to not just $\mathcal{R}_a$ but also $\mathcal{R}_b$ and $\mathcal{R}_c$. Do likewise for SAC-$\mathcal{R}_b$ and SAC-$\mathcal{R}_c$. **[5M]**

2. Plot three separate figures (one for each formulation) for average return vs. timesteps measuring SAC-$\mathcal{R}_i$ evaluated against $\mathcal{R}_i$, $i = \{a, b, c\}$. The y-axis will be different but the x-axis should be the same. How fast does SAC learn with each of these formulations? Is learning faster in some over the others? Explain and justify either ways. **[6M]**

3. Recall that the initial desired behavior for the reacher arm is to reach the target sphere as quickly as possible and stay there. These three reward functions act as different mathematical formulations to achieve this behavior. Now, you need to evaluate which reward formulation(s) do indeed achieve this behavior.

   (a) Take the final learned policy for the three versions. Evaluate it using two metrics:

      i. <u>steps to goal:</u> how many timesteps are required to reach the target?

      ii. <u>steps in target:</u> once the agent reaches the target, how long does it stay there?

   Using the final learned policy, run 500 evaluation episodes where each episode is of length 5000 (as opposed to 1000 previously). Plot a bar chart (with confidence intervals over episodes) showing (i) and (ii) for the three reward formulations. **[4M]**

   (b) Using the experiment in (a), which out of the three reward formulation(s) actually

achieved the desired behavior to the best extent? Explain the results you got. **[7M]**

(c) Plot three separate figures, one for each formulation, showing average return vs. timesteps with SAC-$\mathcal{R}_i$ ($\forall i = \{a, b, c\}$) evaluated separately against $\mathcal{R}_a$, $\mathcal{R}_b$, $\mathcal{R}_c$, respectively.

i. Analyze and comment on each of the three plots. Compare them in terms of learning speed and final performance. **[4M]**

ii. Is there a case when SAC-$\mathcal{R}_j$ performs better/worse than SAC-$\mathcal{R}_i$ when evaluating with respect to $\mathcal{R}_i$ ($i = \{a, b, c\}, j = \{a, b, c\}, j \neq i$)? If yes, what does such a result indicate? Discuss by analyzing the structure of the three reward formulations. **[7M]**

iii. Overall, how do you rate the three reward formulations in terms of (i) ease of specification, (ii) learning efficiency, (iii) achievement of the desired behavior? Which reward formulation(s) would you recommend in practice and why? **[5M]**

3 Learning reward from preferences (bonus)

For many tasks, we may not be able to design an appropriate reward function. It may be misspecified, difficult to design (e.g., cooking), and may require domain expertise. Further, there may be reward exploitation in many cases (recall PA1). One way to overcome this is to learn desired behaviors based on the feedback from humans (teacher). That is, learn a reward function based on the teacher preferences and optimize that reward to obtain a policy. This can avoid the difficulty of reward design because an appropriate feedback can even come from non-experts. This same principle is also used to align LLMs using RLHF.

A popular algorithm for learning from preferences is PEBBLE. It is built on top of SAC and is fairly straightforward to comprehend if you understand the fundamentals of SAC and reward functions. Each preference query corresponds to the agent sampling two trajectories and asking the teacher to label which one is preferred. The reward parameters are learned based on a dataset of many such preference queries. Simultaneously, a policy is optimized over this learned reward function so as to achieve the desired behavior. Know more about PEBBLE from this video.

For the following questions, accompany the results along with the relevant PEBBLE explanation (including the reward loss and update equation).

1. Implement PEBBLE on the Pendulum environment as specified in Section 2.1. Use $\theta_{\text{target}} = \{0, -60, 90, 120, -150\}$. Use a simulated teacher based on the ground-truth reward (check the paper). Plot the results for average return (undiscounted ground-

truth) vs. timesteps as in the paper. You will have to decide on the budget (maximum allowed preference queries). Compare SAC train with ground-truth reward and PEBBLE in terms of learning efficiency and final performance. **[10M]**

2. Run and show experiments for different feedback budgets. Comment on the results. **[5M]**

3. Implement PEBBLE appropriately on the Reacher environment (Section 2.3). Show the results. You will have three types of simulated teachers – each based on one of the three reward formulations. Training with which of the formulation(s) results in faster learning and/or better final performance? **[10M]**